

Sistemas Distribuídos — Aula 30

Revisão Geral do Semestre · Prof. Leandro Borges

Por que revisar o semestre inteiro agora

A Aula 29 foi a apresentação parcial dos trabalhos, com cada grupo mostrando o progresso do Checkpoint 3 antes da reta final. A Aula 30 fecha o conteúdo expositivo do semestre: é a última revisão antes da apresentação final (Aula 31, 15/12) e do encerramento (Aula 32, 17/12). O objetivo aqui não é aprofundar de novo nenhum tema técnico — isso já foi feito nas duas revisões de bimestre, Aula 13 e Aula 27 — mas dar um passo atrás e olhar o semestre inteiro de uma vez: os dois bimestres lado a lado, os grandes temas que atravessam tudo o que foi visto, o elo entre o conteúdo e o trabalho final, e os poucos conceitos que realmente valem a pena dominar bem para quem for revisar por conta própria mais adiante, seja para um TCC, uma prova de proficiência ou uma entrevista técnica.

Mapa do 1º bimestre

O bimestre abriu com fundamentos e metas de projeto (Semanas 1-2): o que caracteriza um sistema distribuído e o vocabulário que sustenta o resto da disciplina — transparência, escalabilidade, confiabilidade, flexibilidade e desempenho. Toda vez que um bloco posterior avalia se uma solução é "boa", é a uma dessas metas que ele está, na prática, se referindo.

Em seguida vieram comunicação entre processos e sincronismo (Semanas 3-4): troca de mensagens entre processos que não compartilham memória, relógios lógicos de Lamport e relógios físicos para ordenar eventos, exclusão mútua distribuída e algoritmos de eleição de coordenador — a base para qualquer coordenação entre nós independentes.

A Semana 5 tratou de cliente-servidor, RPC e RMI: os modelos que escondem a comunicação de baixo nível atrás de uma abstração familiar, seja uma chamada de procedimento remoto (RPC) ou uma chamada de método em um objeto remoto (RMI), incluindo variações como serviço de grupo e comunicação orientada a eventos.

O bimestre fechou com peer-to-peer (Semana 6): o modelo descentralizado, sem servidor central, comparado lado a lado com os demais modelos de comunicação estudados — o primeiro contraponto real à ideia de que todo sistema distribuído precisa de um ponto central de coordenação.

Mapa do 2º bimestre

O segundo bimestre começou com computação em grid e cluster (Aulas 14-15): a ideia de agregar o poder computacional de várias máquinas — de um cluster fisicamente próximo ou de um grid geograficamente distribuído — para viabilizar processamento de alto desempenho que uma única máquina não realizaria sozinha.

Vieram então computação móvel e comunicação em tempo real (Aulas 16-17): como um sistema distribuído lida com nós que mudam de localização, como o roteamento se adapta a essa mobilidade e quais exigências adicionais de latência e continuidade aparecem em aplicações de comunicação on-line.

O núcleo teórico mais denso do bimestre veio com os algoritmos distribuídos avançados (Aulas 18-20): consenso distribuído, snapshot global de Chandy-Lamport e replicação/consistência de dados — os três pilares que sustentam qualquer sistema que precise coordenar múltiplas réplicas.

Na sequência, tolerância a falhas (Aulas 21-23) tratou de como um sistema continua funcionando, ou falha de forma controlada, quando componentes individuais param de responder: modelos de falha, técnicas de detecção, estratégias de replicação e redundância, e procedimentos de recuperação.

Por fim, sistemas operacionais distribuídos (Aulas 24-26) trouxe a camada de infraestrutura que sustenta tudo isso na prática: arquitetura de um SO pensado para múltiplas máquinas, gerência de processos e escalonamento distribuído, e sistemas de arquivos distribuídos que tornam o acesso a dados transparente independentemente de onde estejam armazenados.

Os grandes temas que atravessam o semestre inteiro

Debaixo dos onze blocos de conteúdo dos dois bimestres, quatro ideias se repetem o tempo todo, com uma cara diferente em cada bloco. A primeira é transparência: a ideia de esconder do usuário (ou do programador) que o sistema é distribuído. No 1º bimestre ela apareceu em RPC e RMI, que escondem a chamada remota atrás de uma chamada local; no 2º bimestre, ela reaparece nas arquiteturas de SO distribuído, que escondem a localização real de processos e arquivos.

A segunda é consistência: o que cada réplica de um dado "enxerga" e em que momento. No 1º bimestre, os relógios de Lamport já lidavam com isso ao ordenar eventos sem depender de tempo real; no 2º bimestre, o CAP Theorem e as estratégias de replicação tornam esse trade-off explícito, entre priorizar consistência ou priorizar disponibilidade quando uma partição de rede acontece.

A terceira é tolerância a falhas: a capacidade de um sistema continuar funcionando apesar de falhas parciais. Ela aparece de forma discreta já no modelo cliente-servidor, que precisa lidar com timeout e reenvio de mensagens, e se torna um bloco inteiro no 2º bimestre (Aulas 21-23), com modelos de falha, replicação ativa/passiva, consenso e recuperação.

A quarta é escalabilidade: a capacidade de um sistema crescer sem degradar. No 1º bimestre, o modelo peer-to-peer surge como alternativa ao gargalo de um servidor central; no 2º bimestre, computação em grid e cluster agregam o poder computacional de muitas máquinas, e o escalonamento distribuído de processos organiza esse crescimento na prática.

Quadro-resumo: do conteúdo ao trabalho final

O trabalho final — o sistema distribuído evolutivo desenvolvido em grupo ao longo do semestre — não é um exercício à parte: ele é, na prática, o semestre inteiro aplicado em um projeto só. O Checkpoint 1, entregue em 08/09, corresponde ao bloco de comunicação entre processos do 1º bimestre e exigiu sockets ou RPC/RMI funcionando entre cliente e servidor. O Checkpoint 2, entregue em 29/10, corresponde ao bloco de algoritmos distribuídos do 2º bimestre e pediu múltiplas réplicas do servidor com alguma estratégia de replicação e uma discussão explícita sobre o modelo de consistência adotado — o mesmo trade-off do CAP Theorem, só que aplicado a um sistema real.

O Checkpoint 3, com entrega em 15/12 junto da apresentação final, corresponde ao bloco de tolerância a falhas: detecção de nó fora do ar, failover ou um mecanismo simples de consenso. Quem entende bem os três blocos centrais do semestre — comunicação, replicação/consistência e tolerância a falhas — já entende, essencialmente, a espinha dorsal do próprio trabalho final.

Se for revisar por conta própria, domine estes 6 conceitos

Para quem for revisar o conteúdo da disciplina de forma independente mais adiante — em um TCC, uma prova de proficiência ou uma entrevista técnica — vale a pena priorizar um pequeno núcleo de conceitos em vez de tentar lembrar de tudo com o mesmo grau de detalhe. Os três primeiros são conceituais: transparência de distribuição, que explica o motivo de existir quase todo o resto do conteúdo da disciplina; relógios lógicos de Lamport e ordenação causal, a base para entender qualquer discussão de consistência sem depender de tempo real; e CAP Theorem com os modelos de consistência que dele derivam, porque praticamente todo trade-off de sistema distribuído do mundo real passa por essa escolha entre consistência e disponibilidade.

Os outros três são mais práticos: RPC/RMI como abstração de comunicação, a ponte entre uma chamada de função local e uma chamada remota que é a base de praticamente toda integração entre sistemas distribuídos hoje; modelos de falha e a distinção entre replicação ativa e passiva, que explicam como um sistema real continua de pé quando

algo quebra; e consenso distribuído, com Paxos e Raft como as duas referências mais citadas, que resolvem o problema de como réplicas concordam em um valor mesmo havendo falhas. Dominar bem esses seis conceitos — sabendo defini-los, compará-los e dar um exemplo de cada — cobre a maior parte do que a disciplina realmente quis ensinar.

O que vem a seguir: Aulas 31 e 32

A Aula 31 (15/12) é a apresentação final dos trabalhos: cada grupo apresenta até 10 minutos, com a entrega do Checkpoint 3 completo — código, relatório final com o histórico de decisões de arquitetura e os testes realizados. É também o encerramento do trabalho evolutivo que atravessou o semestre inteiro, dos primeiros sockets do Checkpoint 1 até o tratamento de falhas do Checkpoint 3.

A Aula 32 (17/12) encerra o semestre com devolutivas e fechamento de notas, concluindo o conteúdo das turmas combinadas de Ciência da Computação e Engenharia de Software/Sistemas de Informação. Não há conteúdo novo em nenhuma das duas aulas — o momento de estudo, se for necessário, é agora.

Referências

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. 2ª ed. São Paulo: Pearson Prentice Hall, 2008. — Base teórica do semestre inteiro, do 1º ao 2º bimestre.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T. Sistemas Distribuídos: Conceitos e Projeto. 4ª/5ª ed. Porto Alegre: Bookman. — Referência complementar usada em paralelo em todos os blocos do curso.